

Deep Learning for Solving Dynamic Models

Session 3b: Loss balancing for multi-component residuals

Simon Scheidegger

HEC, University of Lausanne

University of Padova

September 23, 2026

Roadmap of This Lecture

I. Motivation

- ▶ Multi-component losses in PINNs and DEQNs
- ▶ The scale problem, and what it does to the gradient

II. Balancing Methods

- ▶ Equal weighting
- ▶ **Natural units** (the fix to do first)
- ▶ Inverse-loss weighting (+ finger exercise)
- ▶ SoftAdapt, **ReLoBRaLo**, GradNorm

Hands-on: `03_03_Loss_Normalization.ipynb` in `day1/code/03_nas_loss_balancing/`

III. Experiments & Wrap-up

- ▶ Setup and results, three seeds
- ▶ Sensitivity to temperature T
- ▶ The same measurement on a DEQN
- ▶ Connection to PINNs & DSGE
- ▶ Summary and references

Learning objectives

By the end of this lecture you should be able to:

- ▶ Measure gradient dominance: record the gradient norm each weighted loss component sends to the shared parameters
- ▶ Write equilibrium residuals in natural units before any weighting rule is applied, and say why this comes first
- ▶ Implement inverse-loss weighting and ReLoBRaLo in a few lines, and state what each one balances: levels, or progress
- ▶ Read a loss-balancing result as a median over seeds with its range, on a scale-free error
- ▶ Tell a transient scale gap from a permanent one on a DEQN, using the gradient-norm ratio after the transient

Part I

Motivation

Motivation: Multi-Component Losses

Many problems in economics require minimizing *several* loss terms simultaneously:

Physics-informed NNs (PINNs):

- ▶ PDE residual $\mathcal{L}_{\text{PDE}}$
- ▶ Boundary conditions $\mathcal{L}_{\text{BC}}$
- ▶ Initial conditions $\mathcal{L}_{\text{IC}}$

Deep equilibrium nets (DEQNs):

- ▶ Euler equation residuals
- ▶ Budget constraints
- ▶ Market clearing

Multi-task learning:

- ▶ Predict multiple outputs from shared features
- ▶ Each output has its own loss

Core challenge

These loss components often live at **vastly different scales**.

The Scale Problem

Example: three targets of the same shape complexity, at scales 1, 10^2 , 10^4 . Squared errors then sit at

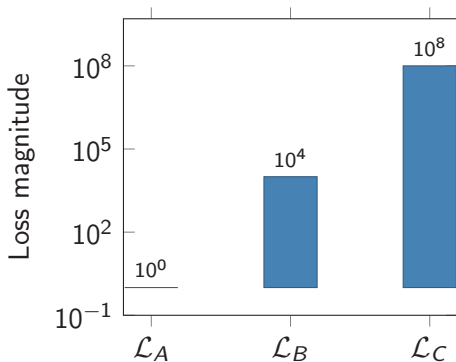
$$\mathcal{L}_A \sim 10^0$$

$$\mathcal{L}_B \sim 10^4$$

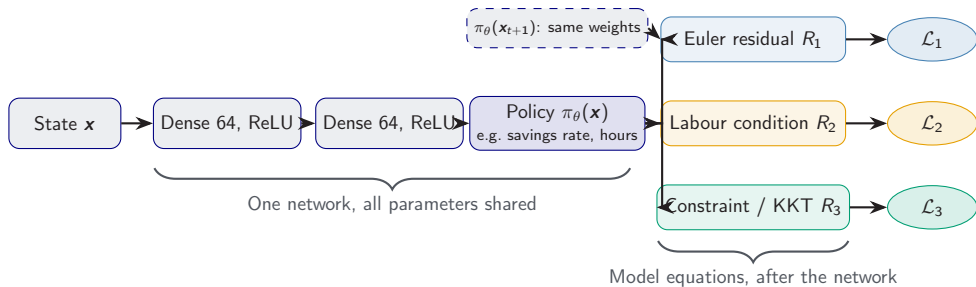
$$\mathcal{L}_C \sim 10^8$$

With equal weights $\mathcal{L}_C$ sends a gradient about $10^6\times$ the size of $\mathcal{L}_A$'s to the shared parameters (the notebook measures $7.0 \cdot 10^3$).

$\Rightarrow$ The network *ignores* the small-scale component.



One Network, Several Equilibrium Conditions



$$\mathcal{L}_{\text{total}} = w_1 \mathcal{L}_1 + w_2 \mathcal{L}_2 + w_3 \mathcal{L}_3, \quad \mathcal{L}_k = \mathbb{E}_x [R_k(x; \theta)^2]$$

The components are equations, not outputs: every residual reads the whole policy, no head owns a loss, and every parameter receives the sum of all three gradients. **How should we set w_1, w_2, w_3 ?**

Loss Balancing Strategies

Equal Weighting

Simplest approach: $w_A = w_B = w_C = 1$

$$\mathcal{L} = \mathcal{L}_A + \mathcal{L}_B + \mathcal{L}_C$$

Problem:

- ▶ If $\mathcal{L}_C \gg \mathcal{L}_A$, gradients are **dominated** by $\mathcal{L}_C$
- ▶ Network learns to minimize $\mathcal{L}_C$ first
- ▶ $\mathcal{L}_A$ barely decreases
- ▶ Small-scale structure is *ignored*

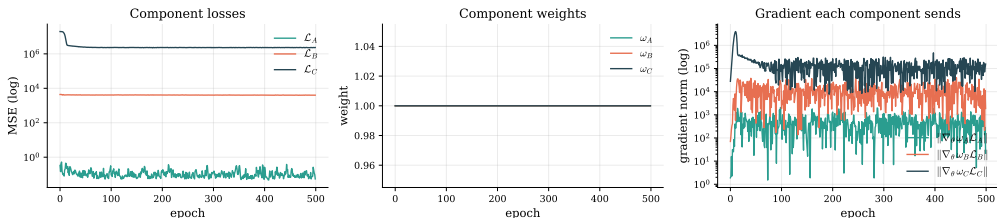
Gradient dominance

$$\frac{\partial \mathcal{L}}{\partial \theta} \approx \frac{\partial \mathcal{L}_C}{\partial \theta} \quad \text{when } \mathcal{L}_C \gg \mathcal{L}_A, \mathcal{L}_B.$$

The network effectively solves a *single-task* problem.

Equal Weighting: Training Dynamics

Equal weighting at scales $10^0, 10^2, 10^4$



Stored teaching run of 03_03, Section 3. Left: component MSE on a log axis. Middle: the constant weights. Right, the panel to read: the norm of the gradient each component sends to the shared parameters, $2.2 \cdot 10^1$, $1.9 \cdot 10^3$, $1.6 \cdot 10^5$ at the end. The update is their sum, so the trunk serves f_C and the other two get what is left. Which of them ends up worst is not the smaller one by default: here it is f_B , at 54% relative error, against 18% for f_A .

Fix 0: Write Each Residual in Its Natural Units

Every equilibrium residual has a scale of its own: a consumption, an output, a marginal utility. Dividing the residual by that scale **before** it enters the loss costs nothing, adds no hyperparameter, and removes the problem at the source.

As coded in Session 5 (05_01, 05_02):

- ▶ Euler equation: divided by λ_t , so the residual is relative to the marginal value of resources
- ▶ Resource constraint: divided by total resources $\sum_j [Y_t^j + (1 - \delta)k_t^j]$
- ▶ Complementarity: Fischer–Burmeister on the unit-free pair $(\mu_t^j / \lambda_t, l_t^j / k_t^j)$

Then the three components are weighted **equally**.

On the toy problem (03_03, Section 4):

	total rel. error
equal weights, scales 1, 10^2 , 10^4	0.861
residuals divided by their scale	0.011

A factor $75\times$ from a change of units. Every adaptive rule in this deck is judged against this line.

Manual Weighting

Idea: Choose weights to approximately equalize the loss magnitudes:

$$\mathcal{L} = w_A \cdot \mathcal{L}_A + w_B \cdot \mathcal{L}_B + w_C \cdot \mathcal{L}_C$$

e.g., $w_A = 10^4$, $w_B = 10^0$, $w_C = 10^{-2}$.

Pros:

- ▶ Can work well if scales are known a priori
- ▶ Simple to implement

Cons:

- ▶ **Trial-and-error:** need to tune d extra hyperparameters
- ▶ **Fragile:** optimal weights change during training as losses evolve
- ▶ **Doesn't generalize:** weights are problem-specific

Natural units first; then, for what remains, a rule that adapts during training.

Inverse-Loss Weighting

Idea: Weight each component inversely proportional to its current magnitude:

$$w_i^{(t)} = \frac{1}{\mathcal{L}_i^{(t)}}$$

This automatically up-weights small losses and down-weights large ones.

With exponential averaging (for stability):

$$\bar{\mathcal{L}}_i^{(t)} = \beta_{\text{ema}} \cdot \bar{\mathcal{L}}_i^{(t-1)} + (1 - \beta_{\text{ema}}) \cdot \mathcal{L}_i^{(t)}$$

$$w_i^{(t)} = \frac{1}{\bar{\mathcal{L}}_i^{(t)} + \epsilon}$$

Pros:

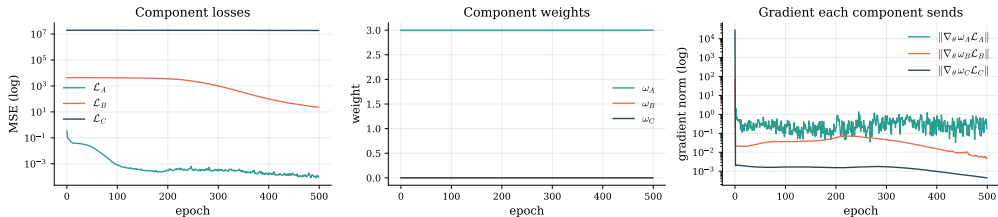
- ▶ Simple, one hyperparameter (β_{ema})
- ▶ Adapts to scales it does not know in advance

Cons:

- ▶ It equalizes the *weighted losses* $w_i \mathcal{L}_i$, not the gradients
- ▶ The gradient of $w_i \mathcal{L}_i$ then scales like $1/e_i$: the component fit **best** pushes hardest
- ▶ Unstable as $\mathcal{L}_i \rightarrow 0$

Inverse-Loss Weighting: The Gradients

Inverse-loss weighting at scales $10^0, 10^2, 10^4$



Stored teaching run of 03_03, Section 5, $\beta_{\text{ema}} = 0.99$, weights normalized to $\sum_i w_i = 3$. The weights end $7.0 \cdot 10^9 \times$ apart and the weighted losses are equal, as designed. The right panel shows the consequence: the gradient goes to whichever component is currently best fit. Total relative error 0.448 against 0.861 for equal weights and 0.011 for natural units.

SoftAdapt

Idea (Heydari et al., 2019): weight by *relative loss change*. A component whose loss is rising, or falling slowly, gets more weight; T sets how sharply.

Weight update

$$\hat{w}_i^{(t)} = K \frac{\exp(\Delta_i^{(t)} / T)}{\sum_j \exp(\Delta_j^{(t)} / T)}, \quad \Delta_i^{(t)} = \mathcal{L}_i^{(t)} - \mathcal{L}_i^{(t-1)}$$

Pros: focuses on the *struggling* component; a few lines of code.

Cons: sensitive to T ; noisy loss changes give noisy weights. Not in the notebook; ReLoBRaLo is its successor.

Scale caveat

Raw Δ_i carries the units of $\mathcal{L}_i$: a component at 10^8 swamps one at 10^0 inside the softmax. Rescale, $\tilde{\Delta}_i = \Delta_i / (\mathcal{L}_i + \varepsilon)$, so the rule reacts to *relative* progress. ReLoBRaLo builds that in.

ReLoBRaLo: Relative Loss Balancing with Random Lookback

Key ingredients (Bischof & Kraus, 2021; CMAME 2025), adapted here as a deterministic classroom version:

1. **Relative balancing:** Use *ratios* $\mathcal{L}_i^{(t)} / \mathcal{L}_i^{(t-1)}$ instead of absolute values
2. **Baseline comparison:** Mix in the **initial losses** $\mathcal{L}_i^{(0)}$ as a stable reference
3. **Exponential smoothing:** Blend new weights with history via α

Why a lookback at all?

Step ratios $\mathcal{L}_i^{(t)} / \mathcal{L}_i^{(t-1)}$ sit near 1 for every component and say nothing about how far each has come. With probability ρ the update compares with the previous step, otherwise with the *initial* loss, so the weights stay anchored to the starting point. The classroom version blends the two deterministically with weight ρ .

ReLoBRaLo: Algorithm

Weight update at epoch t

- 1: Compute losses $\mathcal{L}_1^{(t)}, \dots, \mathcal{L}_K^{(t)}$
- 2: **Step-wise ratios:** $r_{i,s}^{(t)} = \mathcal{L}_i^{(t)} / (T \mathcal{L}_i^{(t-1)} + \varepsilon)$
- 3: **Baseline ratios:** $r_{i,b}^{(t)} = \mathcal{L}_i^{(t)} / (T \mathcal{L}_i^{(0)} + \varepsilon)$
- 4: $\hat{w}_{i,s}^{(t)} = K \text{ softmax}_i(r_{\cdot,s}^{(t)})$, $\hat{w}_{i,b}^{(t)} = K \text{ softmax}_i(r_{\cdot,b}^{(t)})$
- 5: **Combine with smoothing:**

$$w_i^{(t)} = \alpha \left[\rho w_i^{(t-1)} + (1 - \rho) \hat{w}_{i,b}^{(t)} \right] + (1 - \alpha) \hat{w}_{i,s}^{(t)}$$

- ▶ α : smoothing, how much to trust the historical weights
- ▶ ρ : baseline mix, relative weight on the initial-loss comparison
- ▶ T : temperature, sharpness of the softmax

Finger Exercise: Loss Balancing

Exercise

A DEQN has three loss components at iteration t : $\mathcal{L}_1^{(t)} = 10.0$, $\mathcal{L}_2^{(t)} = 0.01$, $\mathcal{L}_3^{(t)} = 1.0$.

1. If you use **equal weights** $w_i = 1/3$, which loss component dominates the gradient? Is this desirable?
2. Compute the **inverse-loss weights**: $w_i \propto 1/\mathcal{L}_i^{(t)}$, normalized to sum to 1.
3. Which component now receives the highest weight? Why does this help training?

Solution: Loss Balancing

Answer

1. With equal weights, $\mathcal{L}_1 = 10.0$ dominates ($100\times$ larger than $\mathcal{L}_2$). The optimizer focuses almost entirely on reducing $\mathcal{L}_1$ while ignoring $\mathcal{L}_2$. This is undesirable if all three conditions must be satisfied simultaneously.
2. Unnormalized: $1/10.0 = 0.1$, $1/0.01 = 100$, $1/1.0 = 1$. Sum = 101.1.
Normalized: $w_1 = 0.001$, $w_2 = 0.989$, $w_3 = 0.010$.
3. $\mathcal{L}_2$ receives the highest weight ($\approx 99\%$), and the three weighted losses are now equal at ≈ 0.01 . That is the whole content of the rule: it equalizes levels. Whether that is what training needs is a different question; the notebook (Section 5) shows the gradient then flows to the component that is currently fit best.

ReLoBRaLo: Hyperparameters

Symbol	Name	Default	Effect
T	Temperature	1.0	Softmax sharpness: small $T \rightarrow$ winner-take-all; large $T \rightarrow$ uniform weights.
α	Smoothing	0.999	Each update moves the weight 0.1% of the way to its target: a time constant of about 1000 <i>updates</i> .
ρ	Baseline mix	0.999	Weight on the initial-loss comparison (convex blend here, Bernoulli in the original).

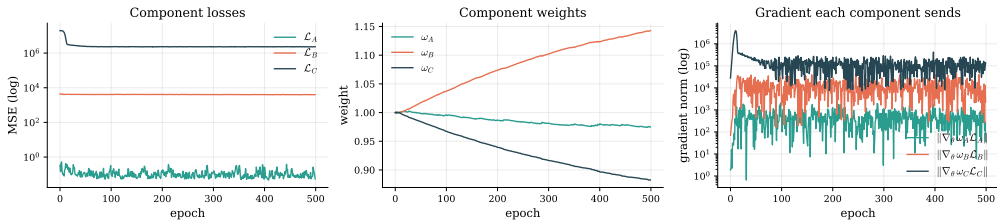
Two ceilings. The softmax separates two weights by at most $e^{\Delta\kappa/T}$ per update, about e at $T = 1$; the smoothing needs about $1/(1 - \alpha)$ updates to get there. Per *epoch*, 500 updates, the weights barely move at any T ; per *optimizer step*, as the authors do, they can. 03_03 runs both.

Practical guidance

Update per step, not per epoch. Non-dimensionalize first; then T hardly matters.

ReLoBRaLo: Weight Evolution

ReLoBRaLo, one update per epoch, $T=1$



Stored teaching run of 03_03, Section 6, one update per epoch, $T=1$. The weights move by $1.29\times$ in 500 updates, and the loss and gradient panels are indistinguishable from equal weighting: total relative error 0.861 against 0.861. Per optimizer step (7,500 updates) the weights reach $1.5\times$ and the total is 0.871. Per step the rule has the authority it was designed with, and it still lands at 0.871 , $76\times$ above natural units: relative balancing is not what a gap in the units needs.

GradNorm: Balance the Gradients Themselves

Idea (Chen et al., 2018): equalize the quantity in the right-hand panels of the last three figures, the gradient norm each weighted component sends to the shared parameters.

Gradient-balance loss

$$\mathcal{L}_{\text{grad}} = \sum_{i=1}^K \left| \left\| \nabla_{\theta} w_i \mathcal{L}_i \right\|_2 - \bar{G} r_i^{(t)} \right|, \quad r_i^{(t)} = \left(\tilde{\mathcal{L}}_i^{(t)} / \text{mean}_j \tilde{\mathcal{L}}_j^{(t)} \right)^{\alpha}, \quad \tilde{\mathcal{L}}_i^{(t)} = \mathcal{L}_i^{(t)} / \mathcal{L}_i^{(0)}$$

$\bar{G}$ is the mean gradient norm; r_i raises the target for the component with the least relative progress ($\alpha = 1.5$ typical). The w_i are trained by gradient descent on $\mathcal{L}_{\text{grad}}$ and renormalized to $\sum_i w_i = K$.

Pros: acts on the gradient, which is what dominance is; can balance a permanent scale gap in principle.

Cheaper cousin for PINNs: Wang, Teng & Perdikaris (2021) set w_i from the ratio of gradient norms directly.

Cons: one extra backward pass per component per update; a second optimizer with its own learning rate and α .

Experiment: Multi-Scale Function Approximation

Experiment Setup

Three target functions on $[0, 1]^2$:

$$f_A(\mathbf{x}) = \text{genz_gaussian}(\mathbf{x}) \\ \text{scale} \in [0, 1]$$

$$f_B(\mathbf{x}) = 100 \times \text{genz_oscillatory}(\mathbf{x}) \\ \text{scale} \in [-100, 100]$$

$$f_C(\mathbf{x}) = 10,000 \times \text{genz_continuous}(\mathbf{x}) \\ \text{scale} \in [0, 10,000]$$

Architecture:

- ▶ Shared trunk: 3×64 ReLU
- ▶ 3 output heads (Dense(1) each)
- ▶ Adam, $\eta = 10^{-3}$

The heads stand in for residuals; what the testbed shares with a DEQN is the trunk.

Data and budget:

- ▶ $n_{\text{train}} = 1,000$, $n_{\text{test}} = 2,000$, $d = 2$
- ▶ 500 epochs, batch 64; three seeds per rule

$$\mathcal{L} = w_A \cdot \text{MSE}(f_A, \hat{f}_A) + w_B \cdot \text{MSE}(f_B, \hat{f}_B) + w_C \cdot \text{MSE}(f_C, \hat{f}_C)$$

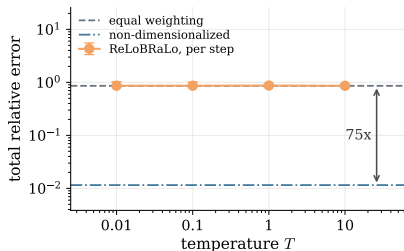
Results: Every Rule Against Natural Units

	Equal	Inverse-loss	ReLoBRaLo per epoch	ReLoBRaLo per step	Natural units (equal weights)
rel. error f_A , seed 0	0.177	0.0082	0.177	0.177	0.0030
rel. error f_B , seed 0	0.540	0.033	0.539	0.539	0.0022
rel. error f_C , seed 0	0.116	0.407	0.116	0.116	0.0031
total, median of 3 seeds	0.861	0.448	0.861	0.871	0.011
range	0.833–0.924	0.418–0.609	0.833–0.933	0.832–0.931	0.0083–0.018

- **Equal:** f_C owns the gradient; the total relative error is 0.861.
- **Inverse-loss:** equalizes the weighted losses; total 0.448, the gradient goes to the best-fit component and f_C pays (0.407 against 0.116).
- **ReLoBRaLo:** per epoch, identical to equal weighting, the smoothing never lets the weights move. Per step, the weights spread $1.5\times$ and the total is 0.871: no better.
- **Natural units:** 0.011, a factor 75 below equal weighting, from a change of units.

Stored teaching run of 03_03, 500 epochs, seeds 0, 1, 2. Relative error = $\overline{|e_i|}/\text{scale}_i$ on 2,000 test points; total is the sum over the three components.

Sensitivity to Temperature T



Stored teaching run of 03_03, Section 7: ReLoBRaLo with per-step updates, three seeds per temperature, median and range of the total relative error against the equal-weighting and natural-units levels.

The gap to natural units is 75 $\times$ or more at every temperature: T is not the lever for a scale gap that sits in the units.

T small ($\ll 1$):

- ▶ Winner-take-all softmax
- ▶ Weights can separate far, if the smoothing lets them

T large ($\gg 1$):

- ▶ Near-equal weights
- ▶ Reverts to the scale problem

Measured: medians from 0.863 ($T = 10$) to 0.871 ($T = 1$); the final weight spread runs from 1.0 $\times$ to 4 $\times$.

The Same Measurement on a DEQN

Brock–Mirman with endogenous labour (02_04, Exercise 2): two policies, two first-order conditions, each in *levels* or in *relative* form. Same 2×64 network, states, optimizer, seed, 3,000 episodes; the gradient norm of each condition recorded every episode.

form	weights	gradient-norm ratio, labour / Euler			final mean rel. error	
		episode 1	episode 300	last	Euler	labour
levels	equal	$2.6 \cdot 10^1$	$3.9 \cdot 10^0$	$3.0 \cdot 10^0$	$2.6 \cdot 10^{-2}$	$1.1 \cdot 10^{-2}$
relative	equal	$1.2 \cdot 10^5$	$3.6 \cdot 10^0$	$1.0 \cdot 10^0$	$2.9 \cdot 10^{-2}$	$6.3 \cdot 10^{-3}$
levels	inverse-loss	$2.6 \cdot 10^1$	$1.2 \cdot 10^0$	$7.4 \cdot 10^1$	$2.5 \cdot 10^{-2}$	$2.1 \cdot 10^{-3}$
relative	inverse-loss	$1.2 \cdot 10^5$	$2.7 \cdot 10^{-1}$	$1.4 \cdot 10^0$	$3.4 \cdot 10^{-3}$	$1.2 \cdot 10^{-1}$

- ▶ A random initial policy violates the labour condition far more than the Euler equation: a gradient-norm gap of $1.2 \cdot 10^5$ at episode 1 in relative form. It is **transient**: the intratemporal condition is easy, and by episode 300 the two send gradients of the same order. Equal weights survive it.
- ▶ Inverse-loss weighting shortens the transient and then keeps acting: in levels the labour error improves to $2.1 \cdot 10^{-3}$ from $1.1 \cdot 10^{-2}$; in relative form the small labour loss gets a near-zero weight and the labour error drifts to $1.2 \cdot 10^{-1}$ from $6.3 \cdot 10^{-3}$. A rule that balances levels cannot tell a solved condition from a neglected one.
- ▶ The Genz gap never closes because it is in the units. Same diagnostic in both cases: the ratio *after* the transient. (Stored teaching run of 03_03, Section 8.)

Connection to PINNs and DSGE

Physics-informed NNs:

- ▶ PDE residual: $\mathcal{L}_{\text{PDE}} \sim \mathcal{O}(1)$
- ▶ Boundary conditions: $\mathcal{L}_{\text{BC}} \sim \mathcal{O}(10^{-2})$
- ▶ Initial conditions: $\mathcal{L}_{\text{IC}} \sim \mathcal{O}(10^{-4})$
- ▶ Rate-balancing rules keep BCs/ICs from being **ignored** once the units agree

DSGE / Deep equilibrium nets:

- ▶ Euler equations, one per agent or country
- ▶ Budget and resource constraints, in goods units
- ▶ Complementarity conditions, in multiplier units
- ▶ Session 5 divides each by its own scale, then weights equally

Where the adaptive rules belong

Bischof & Kraus apply ReLoBRaLo to PINN losses that are already non-dimensionalized, to balance components that converge at different rates. That is the case in the right column after the units are fixed, and the case Session 8 returns to.

Summary

Key takeaways:

1. Gradient dominance is measurable: record $\|\nabla_{\theta} w_i \mathcal{L}_i\|$ per component.
2. **Natural units first.** Free, no hyperparameters, two orders of magnitude on the testbed.
3. Inverse-loss weighting equalizes levels, and sends the gradient to the best-fit component.
4. ReLoBRaLo balances progress: a ceiling of about $e^{1/T}$, reached only with per-step updates.
5. A transient gap closes on its own; a permanent one needs different units, not a rule.

References:

- ▶ Bischof & Kraus (2021). *Multi-Objective Loss Balancing for Physics-Informed Deep Learning*, arXiv:2110.09813; *CMAME* 439, 117914 (2025).
- ▶ Chen et al. (2018). *GradNorm*, ICML.
- ▶ Heydari et al. (2019). *SoftAdapt*, arXiv:1912.12355.
- ▶ Wang, Teng & Perdikaris (2021). *Gradient flow pathologies in PINNs*, *SIAM J. Sci. Comput.* 43(5).

Hands-on

⇒ 03_03_Loss_Normalization.ipynb